

Deep Reinforcement Learning for Algorithmic Trading

Álvaro Cartea¹, Sebastian Jaimungal², Leandro Sánchez-Betancourt³

1 – Oxford University, Mathematical Institute, and Oxford-Man Institute of Quantitative Finance, Email: alvaro.cartea@maths.ox.ac.uk

2 – University of Toronto, Statistical Sciences, Email: sebastian.jaimungal@utoronto.ca

3 – Oxford University, Mathematical Institute, Email: sanchezbetan@maths.ox.ac.uk

We employ reinforcement learning (RL) techniques to devise statistical arbitrage strategies in electronic markets. In particular, double deep Q network learning (DDQN) and a new variant of reinforced deep Markov models (RDMMs) are used to derive the optimal strategies for an agent who trades in a foreign exchange (FX) triplet. An FX triplet consists of three currency pairs where the exchange rate of one pair is redundant because, by no-arbitrage, it is determined by the exchange rates of the other two pairs. We use simulations of a co-integrated model of exchange rates to implement the strategies and show their financial performance.

1.0.1 Learning in financial markets

Continuous-time models and stochastic optimal control techniques are the cornerstone of a great deal of research in algorithmic trading. The design of these models seeks to balance tractability and usefulness. Simple models are ideal to derive trading strategies – in closed-form or as numerical solutions of a system of equations. However, in most cases, these models oversimplify the behaviour of markets to the detriment of the success of the strategies. On the other hand, realistic models are more complex and are often mathematically and computationally intractable with the classical tools of stochastic optimal control.

In this subchapter, we show how to leverage on tools and methods of model-based reinforcement learning (RL) to employ more comprehensive market models, while retaining tractability and interpretability of trading strategies. Our frame-

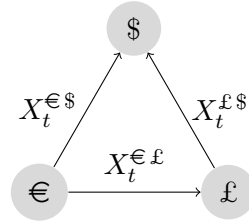


Figure 1: FX triplet. When the trader goes long the pair i/j , she purchases the base currency i (start of the arrow) and pays X^{ij} units of the quote currency j (end of the arrow). Similarly, if the trader sells the pair i/j , she sells the base currency i (start of the arrow) and receives X^{ij} units of the quote currency j (end of the arrow).

work is general and may be applied in all asset classes where instruments trade in an electronic market.

The prices of financial instruments are heteroskedastic, go through regimes of mean-reversion, momentum, and random walks; among other features. While there is a large body of literature on models of financial environments, it is challenging to represent the key features with simple models. For example, in algorithmic trading, the complex dynamics of the limit order book (LOB), including the liquidity taking orders, are generally represented by simple continuous-time models. These models, however, cannot focus on all relevant features of the supply and demand of liquidity, and benefit from only a fraction of the data available to market participants; see (Guéant et al., 2012), Cartea et al. (2014), (Cartea et al., 2015), (Guéant, 2016), (Cartea et al., 2017).

There are three building blocks associated with devising a trading strategy: develop a model of the environment, where states are potentially driven by latent factors; specify a performance criterion, which may include constraints such as risk controls; and, finally, employ techniques to solve a high-dimensional optimisation problem.

Each building block poses challenges and tradeoffs between implementability and accuracy. As a whole, due to the complexity of the trader's objectives, it is very challenging to pose and solve a problem with standard tools of stochastic control in realistic market environments – be it through the dynamic programming principle, stochastic Pontryagin maximum principle, or variational analysis techniques.

In this subchapter, we show how to use RL-based techniques to solve for optimal trading strategies. To streamline the discussion, we focus on foreign exchange (FX) markets. Specifically, we use deep Q-learning and reinforced deep Markov models to trade in an FX triplet. An FX triplet consists of three currency pairs where the exchange rate of one pair is redundant because, by no-arbitrage in a frictionless market (i.e., no fees and zero spread), it is determined by the

exchange rates of the other two pairs.

We consider an agent who trades in a triplet of currency pairs that combine three currencies: EUR, GBP, USD – see Figure 1. When the trader goes long the currency pair GBP/USD, which trades with exchange rate $X_t^{\pounds \$}$, she buys one unit of currency $\pounds$, known as the base currency in the pair, for which she pays $X_t^{\pounds \$}$ units of currency $\$$, known as the quote currency in the pair. Similarly, the cost of one unit of base currency $\pounds$ in the pair EUR/USD is $X_t^{\pounds \$}$ units of the quote currency $\$$; and the cost of one unit of the base currency $\pounds$ in the pair EUR/GBP is $X_t^{\pounds \pounds}$ units of the quote currency $\pounds$. The convention in the FX market is to quote the exchange rate between EUR and USD as EUR/USD, between GBP and USD as GBP/USD, and between EUR and GBP as EUR/GBP. In a frictionless market, one could think of the no-arbitrage exchange rates for the reverse direction of the pairs as $1/X_t^{\pounds \$}$, $1/X_t^{\pounds \pounds}$, $1/X_t^{\pounds \$}$, but these are not quoted in exchanges. A trader who wishes to exchange GBP into USD will sell the currency pair GBP/USD, i.e., consume liquidity from the bid side of the LOB that trades the pair EUR/USD with rate $X_t^{\pounds \$}$; and a trader who wishes to exchange USD into GBP will buy the currency pair GBP/USD, i.e., consume liquidity from the ask side of the LOB for the pair GBP/USD – see (Cartea et al., 2020a).

Therefore, the key identity of a triplet in a frictionless market is the no-arbitrage relationship:

$$X_t^{\pounds \$} X_t^{\pounds \pounds} X_t^{\pounds \$} = 1, \quad (1)$$

so, as mentioned above, the exchange rate of one pair in the triplet is redundant, i.e., the rate of two pairs determines the rate of the third pair.

A classical strategy in FX is to take positions in the three pairs of a triplet when there are arbitrage opportunities – this is commonly known as a triangular arbitrage. The execution of this arbitrage is a mechanical application of a rule-of-thumb that consists of three simultaneous trades to arbitrage the misalignment in the exchange rates of a triplet. These unusual arbitrage opportunities rely on speed advantage and produce riskless profits.

The works (Cartea et al., 2020a,b) study the optimal exchange of a foreign currency into a domestic currency when the currency pair is illiquid. The novelty in the strategies they develop is to trade in an FX triplet (which includes the illiquid pair) to compensate the illiquidity of the one currency pair with two other liquid pairs. The authors show that trading in the triplet considerably enhances the performance of the strategy compared with that of an agent who uses only the illiquid pair to exchange the foreign into the domestic currency. There are several other applications of RL for algorithmic trading in the recent literature, including (Casgrain et al., 2019; Guéant and Manziuk, 2019; Leal et al., 2020; Ning et al., 2018).

In this subchapter, we focus on strategies that are based on the statistical properties of the exchange rates of a triplet.

1.0.2 Statistical arbitrage: trading an FX triplet

We develop a statistical arbitrage strategy for a trader who buys and sells the currency pairs of a triplet over a fixed time horizon T . The success of the strategy stems from taking advantage of co-movements and reversion in the exchange rates of the currency pairs. A statistical arbitrage is not a riskless strategy – the objective of the strategy is to make positive profits (on average), but there is no guarantee that the strategy will not deliver losses.

The trader takes positions in the three currencies of a triplet to maximise a performance criterion given an initial inventory. The trade actions are denoted by $\{a_t^{\mathcal{L}\$}, a_t^{\mathcal{E}\$}, a_t^{\mathcal{E}\mathcal{L}}\}_{t \in \mathbb{T}/\{T\}}$, $\mathbb{T} := \{0, 1, \dots, T\}$. When a_t^{ij} is positive, the trader purchases a_t^{ij} units of the base currency i , for which she pays $X_t^{ij} a_t^{ij}$ units of the quote currency j . When a_t^{ij} is negative, the trader pays a_t^{ij} units of the base currency i and receives $X_t^{ij} a_t^{ij}$ units of the quote currency j . Finally, note that the units of the quantity a^{ij} are those of the currency i .

The inventories in the currencies USD, GBP, EUR, are denoted by $\{q_t^{\$}, q_t^{\mathcal{L}}, q_t^{\mathcal{E}}\}_{t \in \mathbb{T}}$, respectively. It follows that at time t , the inventory in each currency pair is

$$\begin{aligned} q_t^{\$} &= q_0^{\$} - \sum_{j=0}^{t-1} X_j^{\mathcal{E}\$} a_j^{\mathcal{E}\$} - \sum_{j=0}^{t-1} X_j^{\mathcal{L}\$} a_j^{\mathcal{L}\$}, \\ q_t^{\mathcal{L}} &= q_0^{\mathcal{L}} + \sum_{j=0}^{t-1} a_j^{\mathcal{L}\$} - \sum_{j=0}^{t-1} X_j^{\mathcal{E}\mathcal{L}} a_j^{\mathcal{E}\mathcal{L}}, \\ q_t^{\mathcal{E}} &= q_0^{\mathcal{E}} + \sum_{j=0}^{t-1} a_j^{\mathcal{E}\$} + \sum_{j=0}^{t-1} a_j^{\mathcal{E}\mathcal{L}}. \end{aligned}$$

The trader optimises the performance criterion

$$\begin{aligned} \mathbb{E} \left[q_T^{\$} + q_T^{\mathcal{L}} \left(X_T^{\mathcal{L}\$} - \alpha^{\mathcal{L}} q_T^{\mathcal{L}} \right) + q_T^{\mathcal{E}} \left(X_T^{\mathcal{E}\$} - \alpha^{\mathcal{E}} q_T^{\mathcal{E}} \right) \right. \\ \left. - \phi^{\mathcal{L}\$} \sum_{t=0}^{T-1} \left(a_t^{\mathcal{L}\$} \right)^2 - \phi^{\mathcal{E}\$} \sum_{t=0}^{T-1} \left(a_t^{\mathcal{E}\$} \right)^2 - \phi^{\mathcal{E}\mathcal{L}} \sum_{t=0}^{T-1} \left(a_t^{\mathcal{E}\mathcal{L}} \right)^2 \right], \end{aligned} \quad (2)$$

where α^k , for $k \in \{\mathcal{L}, \mathcal{E}\}$, are non-negative liquidation penalty parameters, and ϕ^l , for $l \in \{\mathcal{L}\$, \mathcal{E}\$, \mathcal{E}\mathcal{L}\}$, are non-negative impact parameters that represent the costs, in USD, of walking the limit order book to complete the trade. The units of the terminal penalty parameters α^k are USD/ k^2 , and that of the cost parameters ϕ^l are USD/ l^2 .

On the right-hand side of the performance criterion in (2), the first line shows the USD value of terminal inventory in the three currency pairs. The first term is the inventory in USD, and the other two terms are the terminal inventories in GBP and EUR, which are exchanged into USD with one market order and the

order is subject to a quadratic penalty in the quantity traded with the market order. For example, at the terminal date T , the inventory $q_T^{\mathcal{L}}$ is exchanged into USD at the rate $X_T^{\mathcal{L}\$}$ and the strategy pays the penalty $\alpha^{\mathcal{L}} (q_T^{\mathcal{L}})^2$.

The second line on the right-hand side of (2) shows the total costs from walking-the-book that are paid by the market orders to buy and sell the currency pairs at every point in time until $T - 1$. For example, every time the trader sends a market order for $a^{\mathcal{L}\$}$ units in the currency pair GBP/USD, the costs (in USD) from walking the book are $\phi^{\mathcal{L}\$} (a^{\mathcal{L}\$})^2$.

In the trader's performance criterion, the value of the parameters α and ϕ dictate the severity of the penalties and costs paid by orders sent to the exchange. When the value of the terminal penalty parameter and that of the impact parameter is the same (i.e., $\alpha^{\mathcal{L}} = \phi^{\mathcal{L}\$}$ and $\alpha^{\mathcal{E}} = \phi^{\mathcal{E}\$}$), the costs of executing the orders during the trading window, including the orders executed at T , are the same. On the other hand, if the values of the terminal penalty parameters α^k are very high, relative to the values of ϕ^l , one interprets the terminal penalty paid by the strategy not as a cost of walking the book, but as a penalty that curbs the strategy to reach T with inventories in EUR and GBP very close to or at zero. Thus, if one objective is to reach time T with no inventory in EUR and GBP, one must implement the trading strategy with an arbitrarily high value of α^k to ensure that for all paths of the strategy the terminal inventory q_T^k is zero both in EUR and GBP to avoid the large terminal penalty.

Equivalently, the performance criterion in terms of changes in the exchange rates is

$$\begin{aligned}
 V^a(S_0) := \mathbb{E} \left[\sum_{t=0}^{T-1} \left\{ \left(q_t^{\mathcal{L}} + a_t^{\mathcal{L}\$} - X_t^{\mathcal{E}\mathcal{L}} a_t^{\mathcal{E}\mathcal{L}} \right) \left(X_{t+1}^{\mathcal{L}\$} - X_t^{\mathcal{L}\$} \right) \right. \right. \\
 \left. \left. + \left(q_t^{\mathcal{E}} + a_t^{\mathcal{E}\$} + a_t^{\mathcal{E}\mathcal{L}} \right) \left(X_{t+1}^{\mathcal{E}\$} - X_t^{\mathcal{E}\$} \right) \right\} \right. \\
 \left. - \phi^{\mathcal{L}\$} \sum_{t=0}^{T-1} \left(a_t^{\mathcal{L}\$} \right)^2 - \phi^{\mathcal{E}\$} \sum_{t=0}^{T-1} \left(a_t^{\mathcal{E}\$} \right)^2 - \phi^{\mathcal{E}\mathcal{L}} \sum_{t=0}^{T-1} \left(a_t^{\mathcal{E}\mathcal{L}} \right)^2 \right. \\
 \left. - \alpha^{\mathcal{L}} \left(q_T^{\mathcal{L}} \right)^2 - \alpha^{\mathcal{E}} \left(q_T^{\mathcal{E}} \right)^2 \right]. \quad (3)
 \end{aligned}$$

The equivalence of the two expressions for the performance criterion follows from an inductive argument that we omit for brevity. Below, in the applications, we employ the representation in (3) because the step-by-step rewards of the trading strategy are explicit.

The trader's value function is

$$V(S_0) = \sup_{a \in \mathcal{A}} V^a(S_0) \quad (4)$$

where $S_0 = (X_0^{\mathcal{E}\$}, X_0^{\mathcal{L}\$}, X_0^{\mathcal{L}\mathcal{E}}, q_0^{\mathcal{E}}, q_0^{\mathcal{L}})$, $a = (a^{\mathcal{E}\$}, a^{\mathcal{L}\$}, a^{\mathcal{E}\mathcal{L}})$, and $\mathcal{A}$ is the set of admissible actions.

Below, we simulate the exchange rate dynamics and apply deep Q-learning and reinforced deep Markov models to learn the optimal trading strategies in the three currency pairs of the triplet.

1.0.2.1 Market model

In this subchapter, we simulate market data to derive the trading strategies and gain insights into their behaviour. Specifically, we consider the following cointegrated model of FX rates for two of the currency pairs:

$$X_{t+1}^{\text{€\$}} = X_t^{\text{€\$}} + \kappa_0 \left(\bar{x}^{\text{€\$}} - X_t^{\text{€\$}} \right) + \eta_0 \left(\bar{x}^{\text{£\$}} - X_t^{\text{£\$}} \right) + \epsilon_t^{\text{€\$}}, \quad (5a)$$

$$X_{t+1}^{\text{£\$}} = X_t^{\text{£\$}} + \kappa_1 \left(\bar{x}^{\text{£\$}} - X_t^{\text{£\$}} \right) + \eta_1 \left(\bar{x}^{\text{€\$}} - X_t^{\text{€\$}} \right) + \epsilon_t^{\text{£\$}}, \quad (5b)$$

where $\epsilon_0^{\text{€\$}}, \epsilon_1^{\text{€\$}}, \dots$ and $\epsilon_0^{\text{£\$}}, \epsilon_1^{\text{£\$}}, \dots$ are i.i.d. mean-zero normal random variables. The exchange rate of the third currency pair follows from the no-arbitrage condition in (1).

In the subsequent numerical examples, we employ the following parameters:

$$\bar{x}^{\text{€\$}} = 1.1, \bar{x}^{\text{£\$}} = 1.3, \kappa_0 = \kappa_1 = 0.5, \eta_0 = -0.3, \eta_1 = -0.3,$$

and the standard deviation of the independent shocks $\epsilon^{\text{€\$}}$ and $\epsilon^{\text{£\$}}$ is 0.01. This represents typical weekly FX rate changes, and we consider each time step to be one week in the subsequent analysis.

Although this model does not incorporate the impact of the agent's actions into exchange rates, one can modify it so that the agent's buy and sell activity affects the rates. For example, one can use a propagator model of transient impact, which assumes that the (unobservable) fundamental exchange rates follow (5), while the observed FX rates in the LOB have an additional component due to transient impact. For example, for the currency pair $l \in \{\text{€\$}, \text{£\$}, \text{€£}\}$ assume that the innovation noise in (5) is $\epsilon_t^l \stackrel{\mathbb{P}}{\sim} \mathcal{N}(\eta^\top i_t^l, \sigma^2)$, where $i_{t+1}^{j,l} = \gamma^j i_t^{j,l} + \zeta_t^{j,l}$ with $\zeta_0^{j,l}, \zeta_1^{j,l}, \dots$ i.i.d. mean-zero normal random variables. In the absence of trading, the transient impact factors decay to zero, and the observed FX rates converge to the fundamental ones. Otherwise, exchange rates temporarily drift away from the fundamental rates due to trading actions.

1.0.3 The reinforcement learning paradigm

The objective of RL is to optimise in environments where there is no a priori assumption on the dynamics of the environment and how it is affected by the actions of the agent. A key feature of RL is to “learn” from the interplay between actions and changes in the environment.

In the simplest setting, the evolution of the environment, action, and reward may be viewed as in Figure 2. The the agent's actions a_t affect the transition

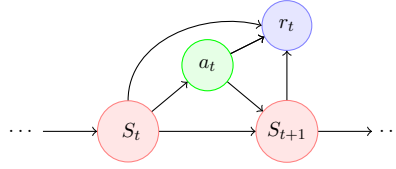


Figure 2: Directed graph representation of the stochastic control problem.

from state S_t into state S_{t+1} . The actions depend only on the state S_t , and the triple (S_t, S_{t+1}, a_t) affect the reward R_t . The collection $[S_t, S_{t+1}, a_t, R_t]$ is known as a 4-tuple. We further assume that the system is Markov.¹

The value of an action policy π (which, in the simplest case, maps states uniquely and deterministically into actions) is given by

$$V^\pi(S) := \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(S_t, S_{t+1}^\pi; a_t^\pi) \mid S_0 = S \right], \quad (6)$$

where a_t^π denotes the agent's action from following policy π , $R(S_t, S_{t+1}^\pi; a_t^\pi)$ denotes the rewards received using action policy π , S_{t+1}^π denotes the one-step ahead state given that the trader takes actions a_t^π when in state S_t , $\gamma \in (0, 1)$ is a discount factor, and the initial state of the system is S .

The **value function** is the value of the best action policy, i.e., the value of the policy that maximises the expectation above, and is written as

$$V(S) := \max_{\pi \in \mathcal{A}} V^\pi(S), \quad (7)$$

where $\mathcal{A}$ is the set of admissible policies.

Due to stationarity, there is no time component in the value function. We restrict the admissible set $\mathcal{A}$ to deterministic policies, so that actions are strictly a function of state and identify the policy π with action a . Under mild conditions, the value function then satisfies the Bellman equation

$$V(S) = \max_{a \in \mathcal{A}} \mathbb{E} [R(S, S^a; a) + \gamma V(S^a)], \quad (8)$$

where the maximisation is taken over a single-action a taken at the current time, S^a denotes the (random) state the system evolves to under this arbitrary action, and $R(S, S^a; a)$ is the (random) reward received from the action a and corresponding state evolution.

A useful concept in learning optimal strategies is the **Q-function**, which measures the *quality* (and hence its name) of an action a at a given point in state space S . The Q-function corresponds to the argument of the max operator in (8), more precisely

$$Q(S, a) := \mathbb{E} [R(S, S^a; a) + \gamma V(S^a)]. \quad (9)$$

¹In the problem formulation above, S_t is the four-dimensional vector $(X_t^{\text{€}}, X_t^{\text{£}}, q_t^{\text{€}}, q_t^{\text{£}})$. The step-by-step rewards are in (3).

```

1 initialize  $Q(s, a)$  and state  $s$  ;
2 for  $i \leftarrow 1$  to  $N$  do
3   select  $\varepsilon$ -greedy action  $a$  from  $Q$ ;
4   observe  $s \xrightarrow{a} (s', R)$  ;
5   update  $Q(s, a) \leftarrow (1 - \beta_k) Q(s, a) + \beta_k \left[ R + \gamma \max_{a' \in \mathcal{A}} Q(s', a') \right]$  ;
6   store  $(s, a, s', R)$  in replay buffer  $\mathcal{D}$ ;
7   for  $j \leftarrow 1$  to  $M$  do
8     select  $(\tilde{s}, \tilde{a}, \tilde{s}', \tilde{R})$  from  $\mathcal{D}$ ;
9     update  $Q(\tilde{s}, \tilde{a}) \leftarrow (1 - \beta_k) Q(\tilde{s}, \tilde{a}) + \beta_k \left[ R + \gamma \max_{\tilde{a}' \in \mathcal{A}} Q(\tilde{S}', \tilde{a}') \right]$  ;
10  end
11  update  $s \leftarrow s'$ ;
12 end

```

Algorithm 1: Dyna-Q learning algorithm.

From (8), the Q-function also satisfies a Bellman-like equation

$$Q(S_t, a) = \mathbb{E} \left[R(S, S^a; a) + \gamma \max_{a' \in \mathcal{A}} Q(S^a, a') \right]. \quad (10)$$

RL focuses on learning this function through interaction with the environment. Tabular Q-learning (Watkins and Dayan, 1992) is one of the classical forms of RL, and entails approximating the state/action space by a discrete set, which renders the Q-function into a large table. The methodology uses the Bellman equation in (10) to obtain updated estimates of the Q-function from actions and observations. Actions are typically ε -greedy; that is, select a random action with probability ε , otherwise select the optimal action based on the current estimate of the Q-function. This allows agents to explore the state/action space, while exploiting what they know so far. After an action, the update of the Q-function is

$$Q(\tilde{s}, \tilde{a}) \leftarrow (1 - \beta_k) Q(\tilde{s}, \tilde{a}) + \beta_k \left[R + \gamma \max_{\tilde{a}' \in \mathcal{A}} Q(\tilde{S}', \tilde{a}') \right], \quad (11)$$

where the parameter set $\{\beta_k\}_{k=1, \dots, N}$ is the learning rate, with $\beta_k > 0$ decreasing, $\sum_{k=1}^{\infty} \beta_k = \infty$, and $\sum_{k=1}^{\infty} \beta_k^2 < +\infty$. Typical examples are $\beta_k = A/(B + k)$ for positive constants $A \geq B$.

A simple way to accelerate learning is to add a so-called *replay buffer* which stores historical 4-tuples (S, S', a, R) corresponding to a state, the action taken, the new state the system evolves to, and the reward received. After taking an action, but before sampling from the environment again, the agent randomly selects 4-tuples from the replay buffer and updates the Q-function. This procedure is known as Dyna-Q learning; see Algorithm 1.

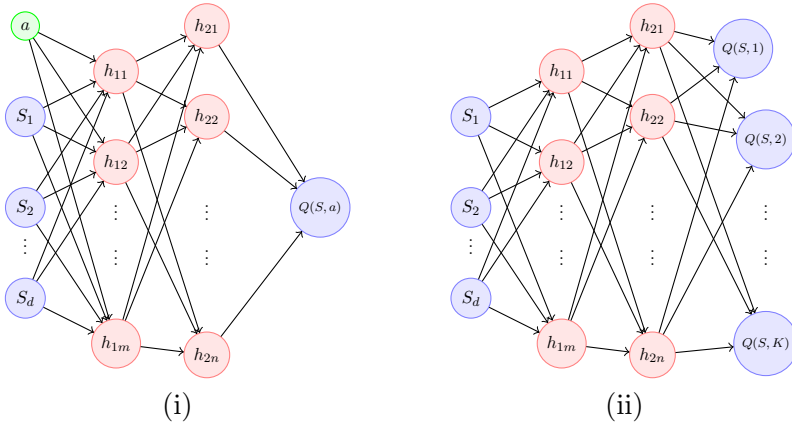


Figure 3: DQN takes either (i) states and actions as inputs and outputs the Q-function value, or, when actions are discrete, (ii) states as inputs and outputs the Q-function for all actions.

1.0.3.1 Deep Q-learning (DQN)

Deep Q-learning (DQN) is conceptually similar to tabular Q-learning, but rather than approximating the state/action space with a discrete space, DQN uses artificial neural nets (ANNs) to approximate the Q-function. The agent assumes that the Q-function is the output of the ANN – the specific architecture of the ANN is arbitrary and should be tuned to the specific problem. Also, the action may be included in the ANN inputs or, if the actions are discrete, the ANN may output the Q-function for all possible action values; see Figure 3. We denote the network parameters by θ . The DQN algorithm (Mnih et al., 2015) proceeds as in Algorithm 1, but the update rules in steps 5 and 9 are replaced by minimising the loss function

$$L(\theta; \theta_T) = \sum_{j=1}^J \left(\left[R_{(j)} + \gamma \max_{a' \in \mathcal{U}_j} Q(S'_{(j)}, a' \mid \theta_T) \right] - Q(S_{(j)}, a_{(j)} \mid \theta) \right)^2 \quad (12)$$

over the network parameters θ . Here, θ_T denotes a target network that is updated to equal the main network θ every M iterations, and $(S_{(j)}, S'_{(j)}, a_{(j)}, R_{(j)})_{j=1, \dots, J}$ corresponds to a mini-batch of 4-tuples. As usual in deep learning, minimising proceeds using gradient descent through back propagation. The loss in (12) uses the optimal actions from the target network rather than the main network θ ; this has been shown to result in sub-optimal strategies.

To improve performance, double deep Q network learning (DDQN) uses the optimal action from the main network instead. Thus, we replace the loss in (12) by

$$L(\theta; \theta_T) = \sum_{j=1}^J \left(\left[R_{(j)} + \gamma Q(S'_{(j)}, a^*_{(j)}(\theta) \mid \theta_T) \right] - Q(S_{(j)}, a_{(j)} \mid \theta) \right)^2, \quad (13)$$

where

$$a_{(j)}^*(\theta) = \operatorname{argmax}_{a' \in \mathcal{U}_j} Q \left(S'_{(j)}, a' \mid \theta \right). \quad (14)$$

Algorithm 2 provides an outline of the procedure in DDQN.

```

1 initialize main and target networks  $\theta, \theta_T$  and state  $s$ ;
2 for  $i \leftarrow 1$  to  $N$  do
3   for  $j \leftarrow 1$  to  $M$  do
4     select  $\varepsilon$ -greedy action  $a$  from  $Q(s, a; \theta)$ ;
5     observe  $s \xrightarrow{a} (s', R)$  and store in replay buffer;
6     grab mini-batch  $J$  from replay buffer;
7     update main network  $\theta$  using SGD on mini-batch loss


$$L(\theta; \theta_T) = \sum_{j=1}^J \left( \left[ R_{(j)} + \gamma Q \left( S'_{(j)}, a_{(j)}^*(\theta) \mid \theta_T \right) \right] - Q \left( S_{(j)}, a_{(j)} \mid \theta \right) \right)^2;$$


     update  $s \leftarrow s'$ ;
8   end
9   update target network  $\theta_T \leftarrow \theta$ ;
10  repeat until converged;
11 end

```

Algorithm 2: Double DQN learning algorithm.

We perform 1,000,000 learning steps of the DQN algorithm with the assumptions and model parameters in Section 1.0.2.1. The remainder of the parameters are: learning rate of the ANN that parameterises the Q-function is $l = 10^{-4}$, replacement frequency $M = 100$, minibatch size of $n_b = 64$, time horizon $T = 10$ weeks, maximum buy/sell quantity is 200,000 units of the base currency at each time step (which we scale down to represent one unit), time step $\Delta_t = 1$ week, discount parameter $\gamma = 0.999$, and the size of the replay buffer is 10,000 previous experiences. The architecture is kept simple and consists of a fully connected feed-forward ANN with two layers with 64 nodes in each layer. We employ the ReLU activation function in both layers and perform update rules according to the Adam optimizer (Kingma and Ba, 2015).

1.0.3.2 Reinforced deep Markov models

Reinforced Deep Markov models (RDMMs), introduced in (Ferreira, 2020) for single asset optimal execution over a few seconds, takes a different approach from that in RL. Instead of approximating the Q-function (see (9)), RDMM proposes a rich graphical model for the environment, actions, and rewards, to capture a wide range of features observed in real markets. In what follows, we first describe deep Markov models (DMMs) (Krishnan et al., 2015), which models the environment's evolution, and then discuss RDMM.

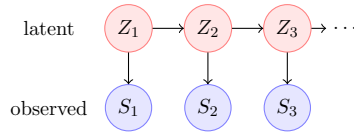


Figure 4: Directed graph representation of a DMM

DMMs may be viewed as stochastic processes that are driven by a latent state which drives the environment and latent states are (randomly) mapped to observable states. Figure 4 shows a graphical model representation of a DMM. The conditional latent state $Z_t|Z_{t-1}$ is independent of $Z_{1:t-2}$ and $S_{1:t-1}$, i.e., Z is Markov, and the conditional observable state $S_t|Z_t$ is independent of all other random variables.² This is a direct generalisation of Hidden Markov models, where the latent state is finite-dimensional. It may also be viewed as a generalisation of Kalman filter models.

In DMMs, the typical conditional one-step evolution of the states is modeled as

$$Z_{t+1}|Z_t \stackrel{\mathbb{P}}{\sim} \mathcal{N}\left(\mu_z^\theta(Z_t); \Sigma_z^\theta(Z_t)\right), \quad \text{latent dynamics,} \quad (15a)$$

$$S_t|Z_t \stackrel{\mathbb{P}}{\sim} \mathcal{N}\left(\mu_s^\theta(Z_t); \Sigma_s^\theta(Z_t)\right), \quad \text{observed data,} \quad (15b)$$

and the means and covariance matrices are parameterised by ANNs with parameters θ . Thus, there are four ANNs, one for each mean vector, and one for each covariance matrix. When the mean ANNs are replaced by affine functions of Z_t , and the covariance matrices are constant, the DMM reduces to a Kalman filter model. The latent state dynamics may be viewed as a time-discretisation of Ito processes, where the instantaneous drift and covariance matrix are given by ANNs: $dZ_t = \mu_z^\theta(Z_t) dt + \sigma_z^\theta(Z_t) dW_t$, where W is a vector of independent Brownian motions and $\sigma_z^\theta(z)$ is the Cholesky decomposition of $\Sigma_z^\theta(z)$. Given a sequence $S_1, S_2, \dots$ of data, one maximises the log-likelihood of observing the sequence of states and rewards (given the actions taken) to estimate the ANN parameters. The log-likelihood, however, is intractable because the posterior distribution of the latent states given data, i.e., $\mathbb{P}(Z_{1:T} | S_{1:T})$, is not attainable in closed-form, nor is it computationally feasible. Instead, in cases like this, we may use variational inference (VI); see (Ormerod and Wand, 2010) for an overview. VI uses an approximate posterior and rather than maximising the log-likelihood, it maximises what is known as the evidence lower bound (ELBO). We elaborate on this approach after we introduce the full RDMM below.

²We use the slice notation $x_{a:b}$ ($a < b$ and integer) to denote the sequence $x_a, x_{a+1}, \dots, x_b$.

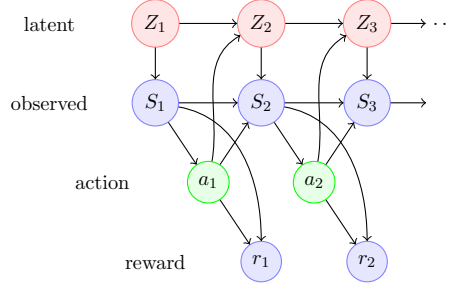


Figure 5: Generative model description of the RDMM framework.

RDMMs are generalised versions of DMMs to incorporate actions and rewards. Here, we adopt the graphical model shown in Figure 5: observable states affect actions, observable states and actions affect rewards, and actions affect the latent and observable states. Thus, there are additional ANNs that connect the various components in the graphical model representation – this architecture is similar to that in (Ferreira, 2020). Here, however, the difference is that there are connections between observables and rewards, rather than between latent states and rewards, and there are connections between observables (which is more reflective of how financial markets work). Moreover, we assume that

$$Z_{t+1}|Z_t, a_t \stackrel{\mathbb{P}}{\sim} \mathcal{N}\left(\mu_z^\theta(Z_t, a_t); \Sigma_z^\theta(Z_t, a_t)\right), \quad \text{latent dynamics,} \quad (16a)$$

$$S_t|Z_t, a_{t-1}, S_{t-1} \stackrel{\mathbb{P}}{\sim} \mathcal{N}\left(\mu_s^\theta(Z_t, S_{t-1}, a_{t-1}); \Sigma_s^\theta(Z_t, S_{t-1}, a_{t-1})\right), \quad \text{observed state,} \quad (16b)$$

$$r_t = R(S_t, a_t), \quad \text{observed reward.} \quad (16c)$$

We further assume that actions are the output of an ANN ϑ with input features given by the observable state

$$a_t = g_a^\vartheta(S_t). \quad (17)$$

One can replace this assumption with a probabilistic model, e.g.,

$$a_t|S_t \stackrel{\mathbb{P}}{\sim} \mathcal{N}\left(\mu_a^\vartheta(S_t); \Sigma_a^\vartheta(S_t)\right), \quad (18)$$

which allows for built-in exploration; we leave such generalisations to future work.

As mentioned earlier, to estimate the DMM that drives the system from observations, we require the posterior distribution of latent states given observations, i.e., we require $\mathbb{P}(Z_{1:T} | S_{1:T}, r_{1:T}, a_{0:T-1})$. This posterior is intractable – both analytically and computationally. Therefore, we adopt the VI approach, and introduce a new probability measure $\mathbb{Q}$ to approximate the posterior distribution

as

$$Z_{t+1}|Z_t, S_{1:T}, r_{1:T}, a_t \stackrel{\mathbb{Q}}{\sim} \mathcal{N}\left(\mu_z^\phi(Z_t, a_t, h_T); \Sigma_z^\phi(Z_t, a_t, h_T)\right). \quad (19)$$

In the above relationship, h_T is a summary of the state sequence $S_{1:T}$ resulting from, e.g., passing $S_{1:T}$ through gated recurrent unit (GRU) (Cho et al., 2014) or long-short-term-memory (LSTM) (Hochreiter and Schmidhuber, 1997) networks. These may also be replaced by their bidirectional versions.

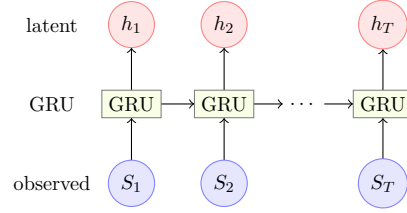


Figure 6: GRU for encoding observations.

The network θ is often referred to as the decoding network (as it maps latent states to observables), while the network ϕ is often referred to as the encoding network (as it maps the observables to latent states).

To estimate model parameters, we maximise the log-likelihood over the decoding network parameters θ . The log-likelihood is intractable, so instead, we use an approximation to the posterior to compute a lower bound for the log-likelihood as follows:

$$\begin{aligned} \mathcal{L}(S_{1:T}, r_{1:T}|a_{0:T-1}) &= \log \mathbb{P}(S_{1:T}, r_{1:T}|a_{0:T-1}) \end{aligned} \quad (20a)$$

$$= \log \int \mathbb{P}(S_{1:T}, r_{1:T}|Z_{1:T}; a_{0:T-1}) d\mathbb{P}(Z_{1:T}|a_{0:T-1}) \quad (20b)$$

$$= \log \int \left(\frac{\mathbb{P}(S_{1:T}, r_{1:T}|Z_{1:T}; a_{0:T-1}) \mathbb{P}(Z_{1:T}|a_{0:T-1})}{\mathbb{Q}(Z_{1:T}|S_{1:T}, a_{0:T-1})} \right) d\mathbb{Q}(Z_{1:T}|S_{1:T}, a_{0:T-1}) \quad (20c)$$

$$\geq \int \log \left(\frac{\mathbb{P}(S_{1:T}, r_{1:T}|Z_{1:T}; a_{0:T-1}) \mathbb{P}(Z_{1:T}|a_{0:T-1})}{\mathbb{Q}(Z_{1:T}|S_{1:T}, a_{0:T-1})} \right) d\mathbb{Q}(Z_{1:T}|S_{1:T}, a_{0:T-1}) \quad (20d)$$

$$= \int \log (\mathbb{P}(S_{1:T}, r_{1:T}|Z_{1:T}; a_{0:T-1})) d\mathbb{Q}(Z_{1:T}|S_{1:T}, a_{0:T-1}) \quad (20e)$$

$$\begin{aligned} & - \int \log \left(\frac{\mathbb{Q}(Z_{1:T}|S_{1:T}, a_{0:T-1})}{\mathbb{P}(Z_{1:T}|a_{0:T-1})} \right) d\mathbb{Q}(Z_{1:T}|S_{1:T}, a_{0:T-1}) \\ & = \mathbb{E}^{\mathbb{Q}} [\log (\mathbb{P}(S_{1:T}, r_{1:T}|Z_{1:T}; a_{0:T-1}))] - KL [\mathbb{Q}(Z_{1:T}|a_{0:T-1}) \parallel \mathbb{P}(Z_{1:T}|a_{0:T-1})], \end{aligned} \quad (20f)$$

where the function KL denotes the Kullback-Leibler divergence (also called relative entropy or information gain), which is a measure of the distance of any given estimated model, with distribution over the data $p_j(x|\hat{\theta}_j)$, to the true model, with distribution over the data $p(x)$. The Kullback-Leibler divergence is defined as

$$KL(p \parallel \hat{p}_j) := \int p(x) \log \frac{p(x)}{p_j(x|\hat{\theta}_j)} dx. \quad (21)$$

The bound in (20f) is called the evidence lower bound (ELBO), so instead of maximising the log-likelihood directly because it is intractable, VI maximises the

ELBO. If the Kullback-Leibler divergence vanishes, then we obtain equality, and the log-likelihood equals the ELBO.

To complete the analysis, we require expressions for each term in (20f). First, from the conditional independence implied by the graphical model in Figure 5 and explicitly modeled in (16), we have

$$\begin{aligned}
 & \log \mathbb{P}(S_{1:T}, r_{1:T} \mid Z_{1:T}, a_{0:T-1}) \\
 &= \sum_{t=1}^T \log \mathbb{P}(S_t, r_t \mid Z_t, a_t, a_{t-1}) \tag{22} \\
 &= -\frac{1}{2} \sum_{t=1}^T \left\{ d_s \log(2\pi) + \log \det \Sigma_s^\theta(Z_t, a_{t-1}) \right. \\
 &\quad + \left(S_t - \mu_s^\theta(Z_t, a_{t-1}) \right)^\top \left(\Sigma_s^\theta(Z_t, a_{t-1}) \right)^{-1} \left(S_t - \mu_s^\theta(Z_t, a_{t-1}) \right) \tag{23} \\
 &\quad + \log(2\pi) + \log \det \Sigma_r^\theta(Z_t, a_t) \\
 &\quad \left. + \left(r_t - \mu_r^\theta(Z_t, a_t) \right)^\top \left(\Sigma_r^\theta(Z_t, a_t) \right)^{-1} \left(r_t - \mu_r^\theta(Z_t, a_t) \right) \right\}.
 \end{aligned}$$

One can estimate the $\mathbb{Q}$ -expectation of the above expression, which appears in the ELBO (20f), as follows: (i) generate $\mathbb{Q}$ -samples of $Z_{1:T}$, (ii) evaluate the above expression, and then (iii) compute the sample average. The Kullback-Leibler term in the ELBO (20f) is

$$\begin{aligned}
 & KL[\mathbb{Q}(Z_{1:T} \mid a_{0:T}) \parallel \mathbb{P}(Z_{1:T} \mid a_{0:T})] \\
 &= \mathbb{E}^{\mathbb{Q}} \left[\log \frac{\mathbb{Q}(Z_{1:T} \mid a_{0:T})}{\mathbb{P}(Z_{1:T} \mid a_{0:T})} \right] \tag{24a}
 \end{aligned}$$

$$= \mathbb{E}^{\mathbb{Q}} \left[\log \frac{\mathbb{Q}(Z_{2:T} \mid Z_1, a_{1:T-1}) \mathbb{Q}(Z_1 \mid a_0)}{\mathbb{P}(Z_{2:T} \mid Z_1, a_{1:T-1}) \mathbb{P}(Z_1 \mid a_0)} \right] \tag{24b}$$

$$= \mathbb{E}^{\mathbb{Q}} \left[\log \frac{\mathbb{Q}(Z_{3:T} \mid Z_2, a_{2:T-1}) \mathbb{Q}(Z_2 \mid Z_1, a_1) \mathbb{Q}(Z_1 \mid a_0)}{\mathbb{P}(Z_{3:T} \mid Z_2, a_{2:T-1}) \mathbb{P}(Z_2 \mid Z_1, a_1) \mathbb{P}(Z_1 \mid a_0)} \right] \tag{24c}$$

$= \dots$

$$= \mathbb{E}^{\mathbb{Q}} \left[\sum_{t=2}^T \log \frac{\mathbb{Q}(Z_t \mid Z_{t-1}, a_{t-1})}{\mathbb{P}(Z_t \mid Z_{t-1}, a_{t-1})} + \log \frac{\mathbb{Q}(Z_1 \mid a_0)}{\mathbb{P}(Z_1 \mid a_0)} \right] \tag{24d}$$

$$\begin{aligned}
 &= \mathbb{E}^{\mathbb{Q}} \left[\sum_{t=2}^T KL[\mathbb{Q}(Z_t \mid Z_{t-1}, a_{t-1}) \parallel \mathbb{P}(Z_t \mid Z_{t-1}, a_{t-1})] \right] \tag{24e} \\
 &\quad + KL[\mathbb{Q}(Z_1 \mid a_0) \parallel \mathbb{P}(Z_1 \mid a_0)].
 \end{aligned}$$

Each term in the sum is the Kullback-Leibler divergence between the $\mathbb{Q}$ and $\mathbb{P}$ one-step transition of the latent factor, conditional on the previous latent state and action. As the conditional distributions involved are all multivariate Gaussian,

the one-step Kullback-Leibler divergence terms may be written explicitly as

$$\begin{aligned}
 & KL[\mathbb{Q}(Z_t | Z_{t-1}, a_{t-1}) \parallel \mathbb{P}(Z_t | Z_{t-1}, a_{t-1})] \\
 &= \frac{1}{2} \left[\log \frac{\det \Sigma_z^\theta(Z_{t-1}, a_{t-1})}{\det \Sigma_z^\phi(Z_{t-1}, a_{t-1}, h_T)} + \text{Tr} \left(\left(\Sigma_z^\theta(Z_{t-1}, a_{t-1}) \right)^{-1} \left(\Sigma_z^\phi(Z_{t-1}, a_{t-1}, h_T) \right) \right) \right. \\
 &\quad \left. + \Delta \mu_z^{\theta, \phi}(Z_{t-1}, a_{t-1}, h_T) \left(\Sigma_z^\theta(Z_{t-1}, a_{t-1}) \right)^{-1} \Delta \mu_z^{\theta, \phi}(Z_{t-1}, a_{t-1}, h_T) - d_z \right], \tag{25}
 \end{aligned}$$

where $\Delta \mu_z^{\theta, \phi}(Z_{t-1}, a_{t-1}, h_T) = \left(\mu_z^\theta(Z_{t-1}, a_{t-1}) - \mu_z^\phi(Z_{t-1}, a_{t-1}, h_T) \right)$. As before, the $\mathbb{Q}$ -expectation in (24e) may be estimated by generating $\mathbb{Q}$ -samples of $Z_{1:T}$, evaluating (25), and then computing the sample average.

Next, we discuss how to obtain the various network parameters in the RDMM approach. The paradigm proceeds in a batch RL manner; it alternates between (i) learn the DMM network while the policy network is held frozen, and (ii) learn the policy network while the DMM network is held frozen. Part (i) of the learning process proceeds in an actor-critic manner: freeze the decoder (generative model) network parameters θ and learn the encoding (approximate posterior) network parameters ϕ by taking SGD steps of the ELBO, then freeze ϕ and use SGD to update θ . Algorithm 3 shows an outline of this procedure.

In simulated environments where actions do not affect the states, or when training with historical data, one may simplify learning as follows: (i) use (simulated or historical) observations and maximise sum of rewards over action network parameters ϑ , and (ii) learn the embedded DMM (encoder ϕ and decoder θ) with the action network held fixed. This last step, however, requires to alternate between maximising over the decoder and encoder networks.

```

1 initialize encoder, decoder, and action networks  $\theta$ ,  $\phi$ , and  $\vartheta$ ;
2 initialize state  $S$ ;
3 for  $i \leftarrow 1$  to  $N$  do
4   for  $t \leftarrow 1$  to  $T$  do
5     select action  $a = g_a^\vartheta(S)$ ;
6     observe  $S \xrightarrow{a} (S', R)$ ;
7     update  $S' \leftarrow S$ ;
8   end
9   simulate  $Z_{1:T}$  using decoder  $\mathbb{Q}$ ;
10  update encoding network  $\theta$  using SGD to maximise ELBO;
11  update decoder network  $\phi$  using SGD to maximise ELBO ;
12  update action network  $\vartheta$  using SGD to maximise total profit;
13 end

```

Algorithm 3: RDMM learning algorithm.

Once the networks are learned using simulated or historical data, live actions may be used to update the trained networks.

1.0.3.3 Implementation of RDMM

Next, we parameterise the ANNs of the RDMM. In a broad sense, there are three types of networks at play: (i) the GRU that encodes the observations, (ii) the ANNs that output the mean vector and the variance-covariance matrices, and (iii) the ANN that models the action.

For (i), we use a standard GRU with input size of two (corresponding to the exchange rates $x^{\text{€\$}}$ and $x^{\text{£\$}}$), five hidden layers, unidirectional, and hidden dimension equal to three. For (ii), we employ a feed-forward ANNs with two layers of 32 nodes each, these networks transform the input into two outputs, the first is the vector of mean values and the Cholesky decomposition of the variance-covariance matrix of the multivariate normal. To model an $n \times n$ variance-covariance matrix, the ANN outputs an $n(n+1)/2$ -dimensional vector that characterises the lower triangular matrix in the Cholesky decomposition of the variance-covariance matrix. We reshape the $n(n+1)/2$ -dimensional vector into a lower triangular matrix and ensure that the values in the diagonal are positive which is a sufficient condition for the lower-triangular matrix to be positive-definite – (Dorta et al., 2018).

Finally, for (iii) we use a feed-forward ANN with two layers of 32 nodes each, input dimension being equal to five ($t, X^{\text{€\$}}, X^{\text{£\$}}, q^{\text{€}}, q^{\text{£}}$), and output dimension equal to three ($a^{\text{€\$}}, a^{\text{£\$}}, a^{\text{€£}}$). To make the actions lie between a maximum and minimum range, we scale the output of the last layer, for which we use a $\tanh(x)$ activation. In the three cases: (i), (ii), (iii), we employ ReLU activations in the intermediate layers, and perform update rules according to the Adam optimizer (Kingma and Ba, 2015).

We run over 100,000 learning steps on each network in batches of 64 sample paths at a time, and 64 simulations to compute the $\mathbb{Q}$ -expectations. Figure 7 shows the loss function (left panel), and the value of the rewards (right panel) as a function of the learning steps. Recall that the loss function in (20f) is the result of two calculations, the $\mathbb{Q}$ -expectation of (23) and the Kullback-Leibler term in (24e), for which each term is explicitly given in (25).

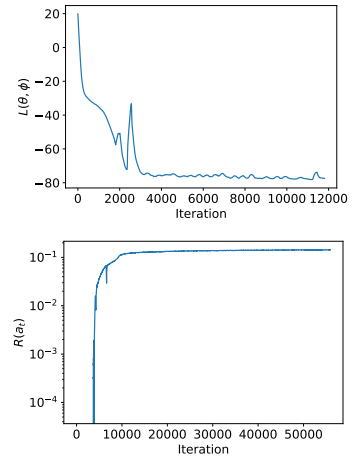


Figure 7: Loss function and expected reward: moving average over 100 iterations.

1.0.4 Optimal trading in triplet

The three-dimensional optimal strategy has various features – we discuss them below. We show plots only for the RDMM model because the results for DQN

are similar. The walk-the-book cost parameters are $\phi^{\text{€\$}} = \phi^{\text{£\$}} = \phi^{\text{€£}} = 0.001$, and the terminal penalty parameters are $\alpha^{\text{€}} = \alpha^{\text{£}} = 1$ – see (2).

First, we study the optimal action through time as a function of the inventory. The state space is five-dimensional $(t, X_t^{\text{€\$}}, X_t^{\text{£\$}}, q_t^{\text{€}}, q_t^{\text{£}})$ and the output actions is three-dimensional $(a_t^{\text{€\$}}, a_t^{\text{£\$}}, a_t^{\text{€£}})$, thus, for each plot in this section we fix several inputs and we depict the optimal strategy. Figure 8 displays the optimal strategy $a_t^{\text{€\$}}$ learnt by the RDMM when $t = 5$ weeks (left panel), $t = 7$ weeks (middle panel), $t = 9$ weeks (right panel), and the terminal date is $T = 10$ weeks. In the three plots: $q_t^{\text{£}} = 0$, $X_t^{\text{£\$}} = 1.3$, the x -axis is the inventory $q_t^{\text{€}}$, and the y -axis is the exchange rate $X_t^{\text{€\$}}$. We observe that the optimal strategy learns that as the trader approaches T , the inventory should be close to zero to avoid the terminal penalty. Note that the value of the terminal penalty parameter α^k is greater than that of the cost parameter ϕ^l . The plots for the learnt actions $a_t^{\text{£\$}}$ and $a_t^{\text{€£}}$ are similar.

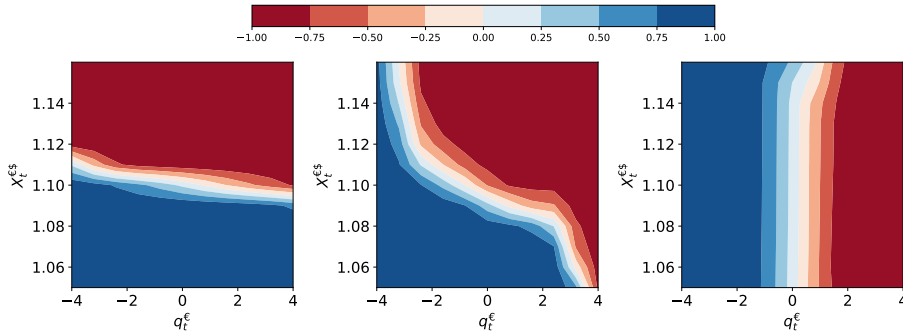


Figure 8: Optimal action $a_t^{\text{€\$}}$ as a function of $q_t^{\text{€}}$ and $X_t^{\text{€\$}}$. Left panel $t = 5$, middle panel $t = 7$, and right panel $t = 9$. Trading horizon $T = 10$ weeks.

In Figure 9 we set $t = 5$ weeks and let $X_t^{\text{£\$}} = 1.33$ in the top row (above its mean-reverting level), $X_t^{\text{£\$}} = 1.30$ in the middle row (equal to its mean-reverting level), and $X_t^{\text{£\$}} = 1.27$ in the bottom row (below its mean-reverting level). In all plots, the x -axis represents the inventory in Euros, $q_t^{\text{€}}$, and the y -axis represents the exchange rate for the pair EUR/USD, $X_t^{\text{€\$}}$. The colours represent the optimal actions learnt by the RDMM: left, middle, right columns show the actions $a_t^{\text{€\$}}$, $a_t^{\text{£\$}}$, $a_t^{\text{€£}}$, respectively. When the rate $X_t^{\text{£\$}}$ is below its mean-reverting level, the optimal action $a_t^{\text{£\$}} = 1$ (i.e., buy GBP and sell USD) is almost independent from the level of the rate $X_t^{\text{€\$}}$ and from the inventory $q_t^{\text{€}}$ (see bottom middle panel). Similarly, when the rate $X_t^{\text{£\$}}$ is above its mean-reverting level, the optimal action $a_t^{\text{£\$}} = -1$ (i.e., sell GBP and receive USD) is almost independent from the rate $X_t^{\text{€\$}}$ and from the inventory $q_t^{\text{€}}$. Thus, the optimal strategy learnt by the RDMM provides the trader with a “buy low, sell high” strategy.

The middle panel shows that when the rate $X_t^{\text{€\$}}$ is below (above) its mean-

reverting value, the value of EUR in units of USD and, also, in units of GBP, are both, under-priced (over-priced). Thus, the actions $a_t^{\text{€}\$}$ and $a_t^{\text{€}\text{£}}$ are greater (smaller) than zero, i.e., the strategy buys (sells) EUR low (high).

The plot in the centre requires a more careful analysis. The equations in (5) show the effect that $\bar{x}^{\text{€}\$} - X_t^{\text{€}\$}$ has on $X_{t+1}^{\text{£}\$}$. If $\bar{x}^{\text{€}\$} - X_t^{\text{€}\$}$ is greater (smaller) than zero and $\bar{x}^{\text{£}\$} - X_t^{\text{£}\$} = 0$, then, on average, $X_{t+1}^{\text{£}\$}$ is greater (smaller) than $X_t^{\text{£}\$}$. Thus, when $X_t^{\text{£}\$} = \bar{x}^{\text{£}\$}$, the strategy learns that the optimal action $a_t^{\text{£}\$}$ is to buy $X_t^{\text{£}\$}$ if $X_t^{\text{€}\$} > \bar{x}^{\text{€}\$}$ and sell $X_t^{\text{£}\$}$ if $X_t^{\text{€}\$} < \bar{x}^{\text{€}\$}$, because, on average, the trader makes a profit.

Finally, in the top row (bottom row), when the exchange rate $X_t^{\text{£}\$}$ is above (below) its mean-reverting value, the optimal strategy raises (lowers) the threshold for when to buy/sell when compared with the threshold in the middle row. The RDMM learns the dynamics of the environment, so when $X_t^{\text{£}\$} = 1.33$ or $X_t^{\text{£}\$} = 1.27$, the regions where $X_{t+1}^{\text{€}\$}$ is, on average, above/below $X_t^{\text{€}\$}$, changes to those shown in the figure. For example, from equation (5) we see that, on average, $X_{t+1}^{\text{€}\$} > X_t^{\text{€}\$}$, if and only if $X_t^{\text{€}\$} < \bar{x}^{\text{€}\$} + \eta_0/\kappa_0 (\bar{x}^{\text{£}\$} - X_t^{\text{£}\$})$, so the threshold in the top left panel is around 1.118 ($X_t^{\text{£}\$} = 1.33$), and in the bottom left panel, the threshold is around 1.082 ($X_t^{\text{£}\$} = 1.27$) – recall that $\eta_0 = -0.3$, $\kappa_0 = 0.5$, $\bar{x}^{\text{€}\$} = 1.1$.

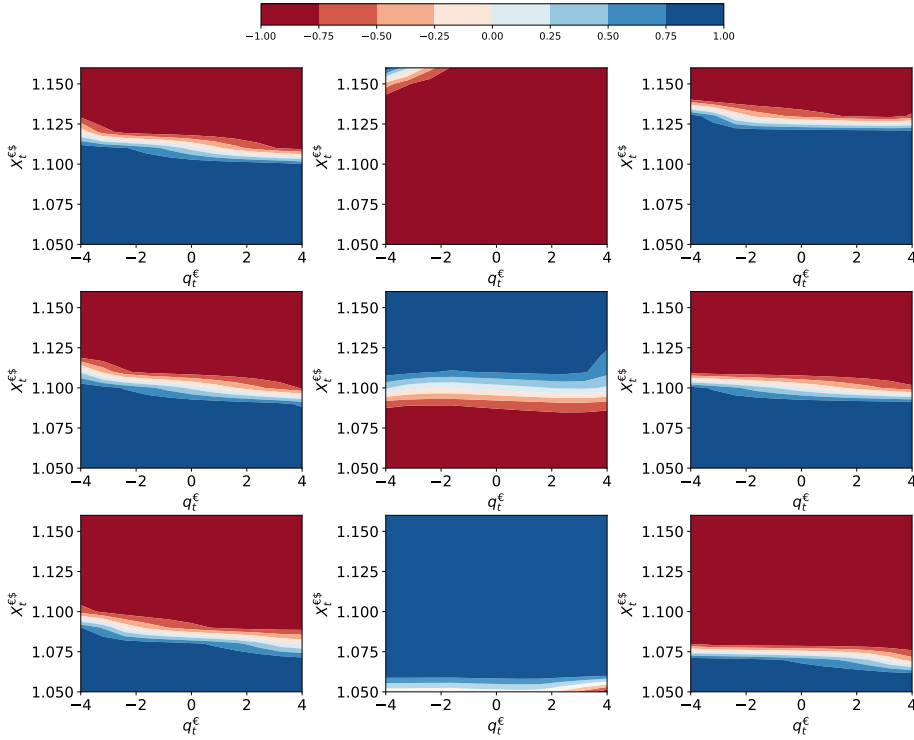


Figure 9: Left column: optimal action $a_t^{\text{€}\$}$; middle column: optimal action $a_t^{\text{£}\$}$; right column: optimal action $a_t^{\text{€}\text{£}}$. Top row: $X_t^{\text{£}\$} = 1.33$; middle row: $X_t^{\text{£}\$} = 1.30$; bottom row: $X_t^{\text{£}\$} = 1.27$. Mean-reverting levels in EUR/USD and GBP/USD are $\bar{x}^{\text{€}\$} = 1.1$ and $\bar{x}^{\text{£}\$} = 1.3$.

Next, we discuss the performance of the optimal actions in the triplet. We perform 10,000 simulations of the exchange rate process and execute the optimal actions learnt by the RDMM. In all simulations, the rate processes follow (5), and the trader starts with zero inventory in all currencies, i.e., $q_0^{\text{€}} = q_0^{\text{£}} = q_0^{\text{£}\$} = 0$. Figure 10 shows a histogram of the profit and loss in USD. It is not surprising to see that in such a controlled environment, the optimal actions lose money in less than 1% of the runs because the strategies fully exploit the learnt dynamics of the environment. The mean value of the P&L is 0.14 USD and the standard deviation

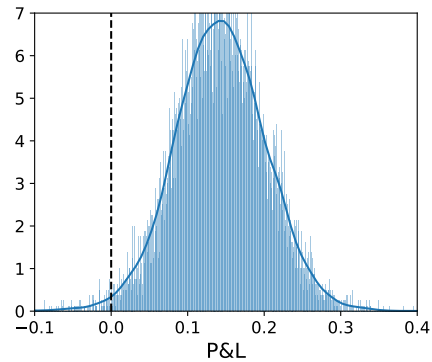


Figure 10: Profit and loss in USD.

tion is 0.06. Here, 0.14 USD is the gain when the maximum quantity we buy/sell is one. Recall that we think of each trade of being of size 200,000 of the base currency, thus, the P&L of the strategy is around 28,000 USD over the course of ten weeks.

Figure 11 displays the paths for $q_{0:T}^{\text{€}}$ and $q_{0:T}^{\text{£}}$. The shaded region encompasses the data between the 10% and 90% quantiles, and the red line is one of the inventory paths. We observe that the strategy performs statistical arbitrages until approximately time $t = 7$ and then liquidates any outstanding inventory in EUR and GBP to finish with a flat inventory in those two currencies at time T . Had we taken the values of the impact parameters to be equal to the values of terminal inventory parameters, then we would not observe the concentration at zero of $q_T^{\text{€}}$ and $q_T^{\text{£}}$ because the strategy would in many runs find it optimal to liquidate inventory at the end of the trading horizon.

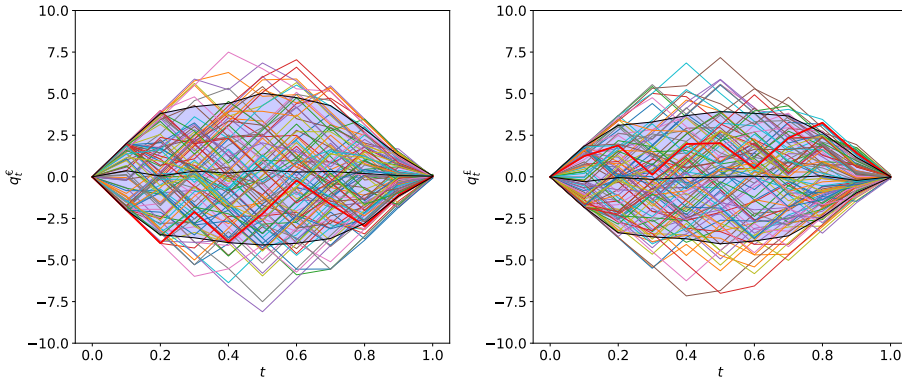


Figure 11: Inventory paths for $q_T^{\text{€}}$ and $q_T^{\text{£}}$. The shaded region shows the 10%, 50%, and 90% quantiles through time.

The top row in Figure 12 shows stylised features of the optimal actions $a_{0:T}^{\text{€\$}}$, $a_{0:T}^{\text{£\$}}$, $a_{0:T}^{\text{€£}}$ in the currency pairs $X_{0:T}^{\text{€\$}}$, $X_{0:T}^{\text{£\$}}$, $X_{0:T}^{\text{€£}}$, respectively, for one simulation. The green up-arrows are buys and the red down-arrows are sells. The bottom row shows the inventories $q_{0:T}^{\text{€}}$ and $q_{0:T}^{\text{£}}$, and the cash process $q_{0:T}^{\text{\$}}$. This figure highlights the interplay between buy-low-sell-high and inventory control. As shown in the bottom panels, the trader aims to finish with $q_T^{\text{€}} \approx 0$ and $q_T^{\text{£}} \approx 0$ while taking advantage of her learnt knowledge about the exchange rate dynamics – see that most of the green arrows are below the dotted lines (mean-reverting level of the currency pairs), and, most of the red arrows are above the dotted lines.

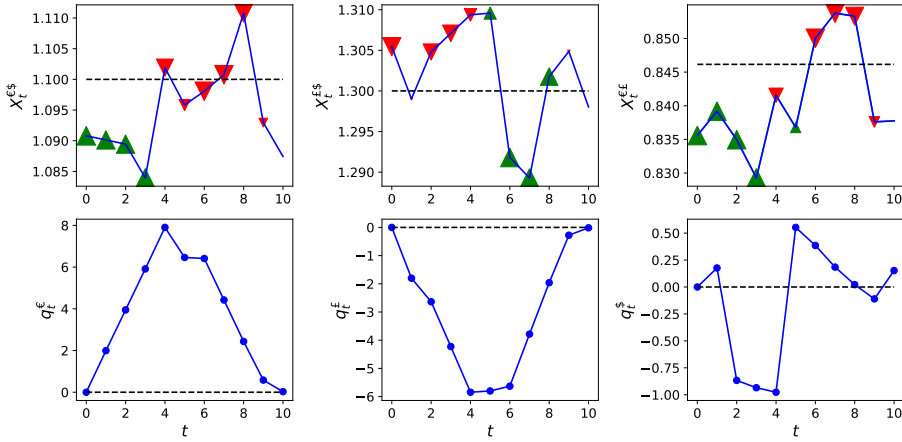


Figure 12: RDMM optimal strategy sample path showing FX rates, actions (size and direction of arrows), and currency inventories.

1.0.4.1 Remarks on RDMM versus DDQN

Next, we briefly look at the DDQN results. The DDQN and RDMM approaches produce similar optimal actions, however, as we see from comparing Figures 13 and 8, the actions from DDQN tend to be noisier and generate some unusual behaviour compared with the actions from RDMM. Moreover, the action space in RDMM can be discrete or continuous, while DDQN seeks optimal strategies over a discrete action space and tends to work only when the action space is small. Also, learning in RDMM from historical data benefits from augmenting the historical data set with simulations from the learned DMM portion of the RDMM. On the other hand, DDQN is limited to historical data; however, one may use a replay buffer to help stabilise the results. Thus, RDMM produces more stable action networks than those from the DDQN.

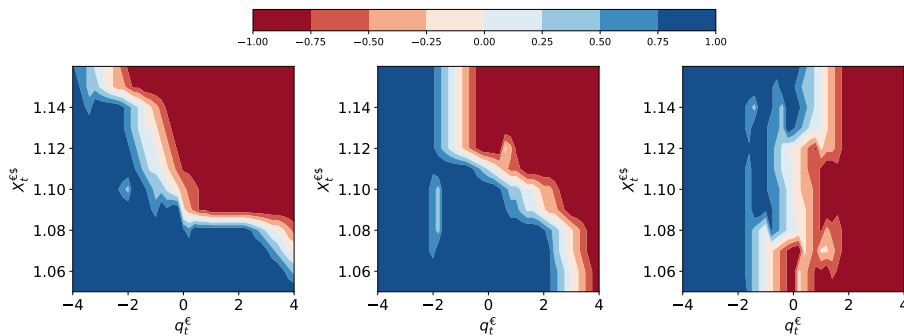


Figure 13: Optimal action $a_t^{\text{€\$}}$ as a function of $q_t^{\text{€}}$ and $X_t^{\text{€\$}}$. Left panel $t = 5$, middle panel $t = 7$, and right panel $t = 9$. Trading horizon $T = 10$ weeks.

1.0.5 Conclusions and future work

We showed how to devise a statistical arbitrage strategy that takes advantage of co-movements and mean-reversion in the exchange rates of an FX triplet. Through simulations, we illustrated the financial performance of the trading strategy based on the RDMM approach. Our approach can be applied in all asset classes where instruments are traded in electronic markets.

To benefit from all the features of RDMM, one needs to expand our study with market data. RDMM can learn a generative model for the data, which is used to simulate more data (with the same statistical properties of the original data). This can play a crucial role in the learning of optimal strategies when data are limited.

One can implement the strategies derived throughout this subchapter in live trading. For this, one proceeds as in stochastic control frameworks: learn the network parameters offline (this is equivalent to computing the value function in stochastic control); employ the learnt network parameters live to compute the optimal action. In principle, one can also update the network while new information arrives; however, this approach is computationally expensive, and instead it may be preferable to incorporate new data to update the networks in advance, e.g., overnight or a few hours before executing the strategy.

Bibliography

- Cartea, Á., Donnelly, R., and Jaimungal, S. (2017). Algorithmic trading with model uncertainty. *SIAM Journal on Financial Mathematics*, 8(1):635–671.
- Cartea, Á., Jaimungal, S., and Jia, T. (2020a). Trading foreign exchange triplets. *SIAM Journal on Financial Mathematics*, 11(3):690–719.

- Cartea, Á., Jaimungal, S., and Penalva, J. (2015). *Algorithmic and High-Frequency Trading*. Cambridge University Press.
- Cartea, Á., Jaimungal, S., and Ricci, J. (2014). Buy low, sell high: A high frequency trading perspective. *SIAM Journal on Financial Mathematics*, 5(1):415–444.
- Cartea, Á., Perez Arribas, I., and Sánchez-Betancourt, L. (2020b). Optimal execution of foreign securities: A double-execution problem with signatures and machine learning. *Available at SSRN*.
- Casgrain, P., Ning, B., and Jaimungal, S. (2019). Deep Q-learning for Nash equilibria: Nash-DQN. *arXiv preprint arXiv:1904.10554*.
- Cho, K., van Merriënboer, B., Bahdanau, D., and Bengio, Y. (2014). On the properties of neural machine translation: Encoder–decoder approaches. *Syntax, Semantics and Structure in Statistical Translation*.
- Dorta, G., Vicente, S., Agapito, L., Campbell, N. D., and Simpson, I. (2018). Structured uncertainty prediction networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 5477–5485.
- Ferreira, T. A. (2020). Reinforced deep markov models with applications in automatic trading. *arXiv preprint arXiv:2011.04391*.
- Guéant, O. (2016). *The financial mathematics of market liquidity: From optimal execution to market making*, volume 33. CRC Press.
- Guéant, O., Lehalle, C.-A., and Fernandez-Tapia, J. (2012). Optimal portfolio liquidation with limit orders. *SIAM Journal on Financial Mathematics*, 3(1):740–764.
- Guéant, O. and Manziuk, I. (2019). Deep reinforcement learning for market making in corporate bonds: beating the curse of dimensionality. *Applied Mathematical Finance*, 26(5):387–452.
- Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8):1735–1780.
- Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. In Bengio, Y. and LeCun, Y., editors, *3rd International Conference on Learning Representations, ICLR 2015*.
- Krishnan, R. G., Shalit, U., and Sontag, D. (2015). Deep Kalman filters. *arXiv preprint arXiv:1511.05121*.
- Leal, L., Laurière, M., and Lehalle, C.-A. (2020). Learning a functional control for high-frequency finance. *arXiv preprint arXiv:2006.09611*.

- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533.
- Ning, B., Lin, F. H. T., and Jaimungal, S. (2018). Double deep Q-learning for optimal execution. *arXiv preprint arXiv:1812.06600*.
- Ormerod, J. T. and Wand, M. P. (2010). Explaining variational approximations. *The American Statistician*, 64(2):140–153.
- Watkins, C. J. and Dayan, P. (1992). Q-learning. *Machine learning*, 8(3-4):279–292.